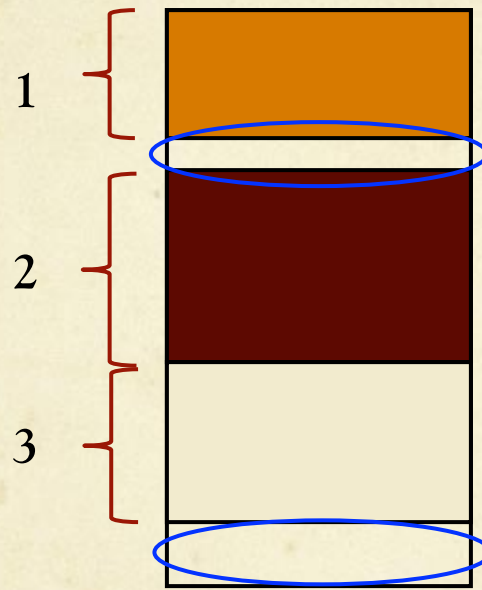


External fragmentation

- ◆ Reduces memory *utilization*
 - ◆ Even if enough free memory *exists* to allocate new process...
 - ◆ ...not useful since memory is not *contiguous*
- ◆ Possible approaches
 - ◆ Reduce external fragmentation using *compaction*
 - ◆ Find way to allow *non-contiguous* partitions
 - ◆ I.e., split process footprint into multiple parts

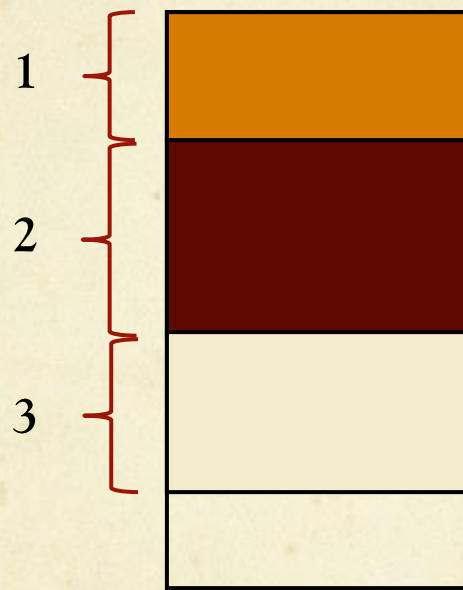
Compaction



Partitioned memory

- ◆ Move partitions to abut each other

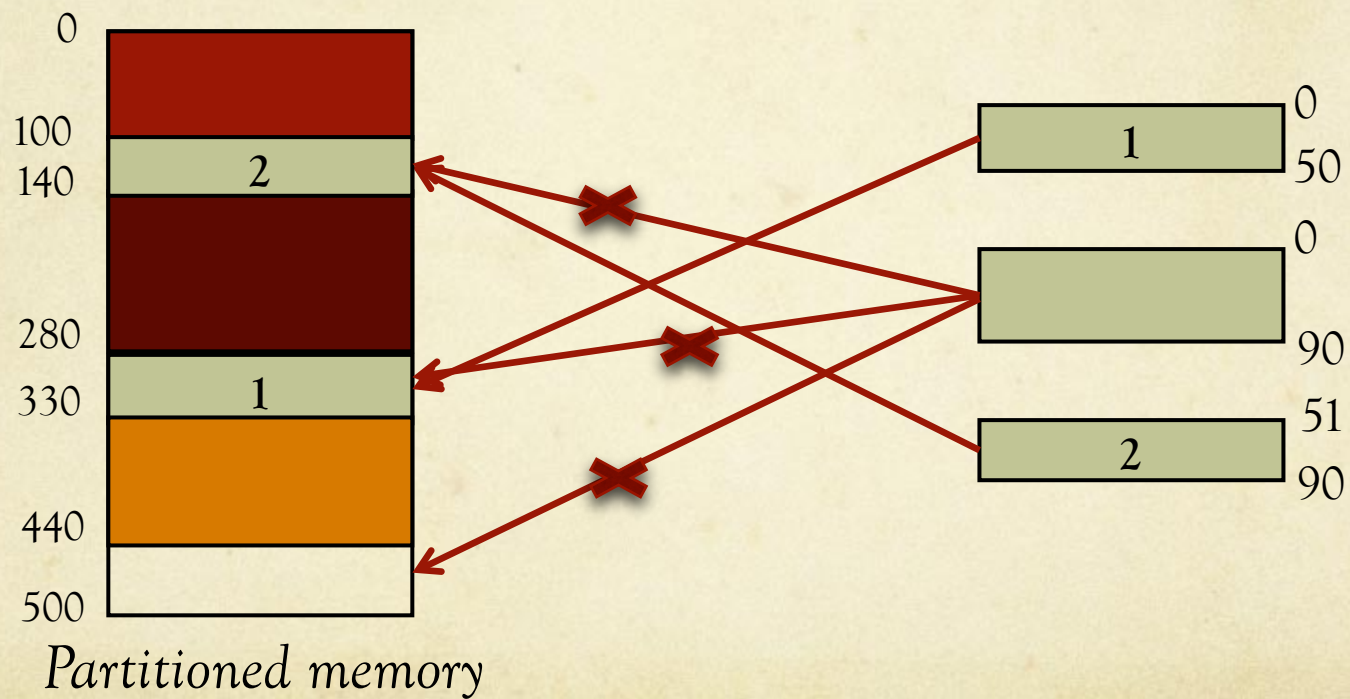
Compaction



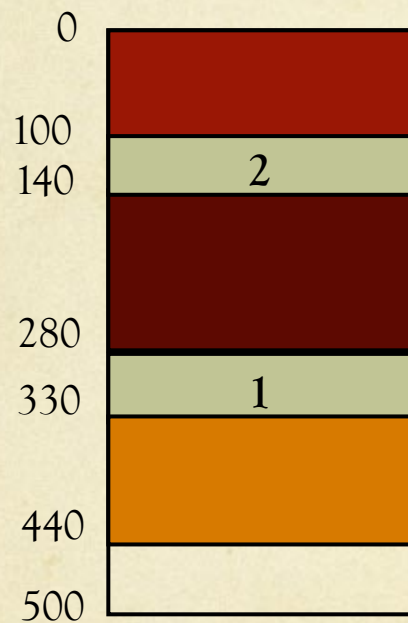
Partitioned memory

- ◆ Move partitions to abut each other
- ◆ Needs partition *relocation*

Allowing non-contiguous allocation...



Allowing non-contiguous allocation...



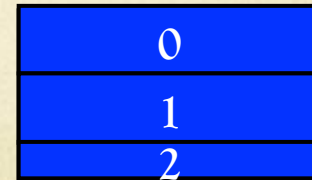
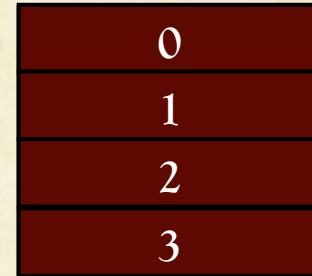
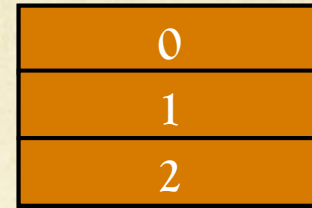
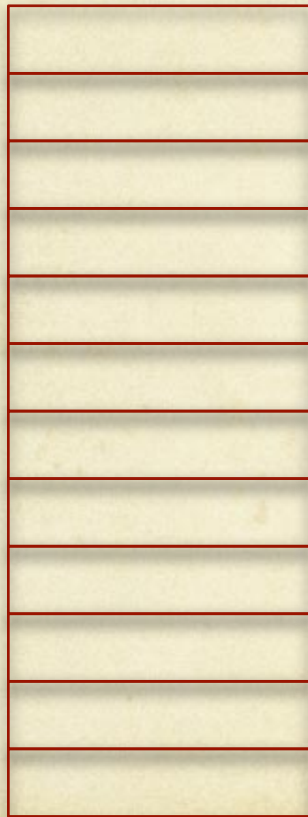
Partitioned memory

- ◆ What is the *base address*?
- ◆ What is the *limit*?

What we need...

- ◆ ...is a more *sophisticated* memory management mechanism!
 - ◆ Maintain information for *multiple* parts of process
- ◆ One such mechanism is **paging**
- ◆ Divide physical memory into equal-sized blocks: *page frames*
- ◆ Divide logical memory into same sized blocks: *pages*
- ◆ Process pages *mapped* to physical page frames
- ◆ Page frame sizes are powers of *2*
 - ◆ E.g., 4096 bytes, 8192 bytes

Paging



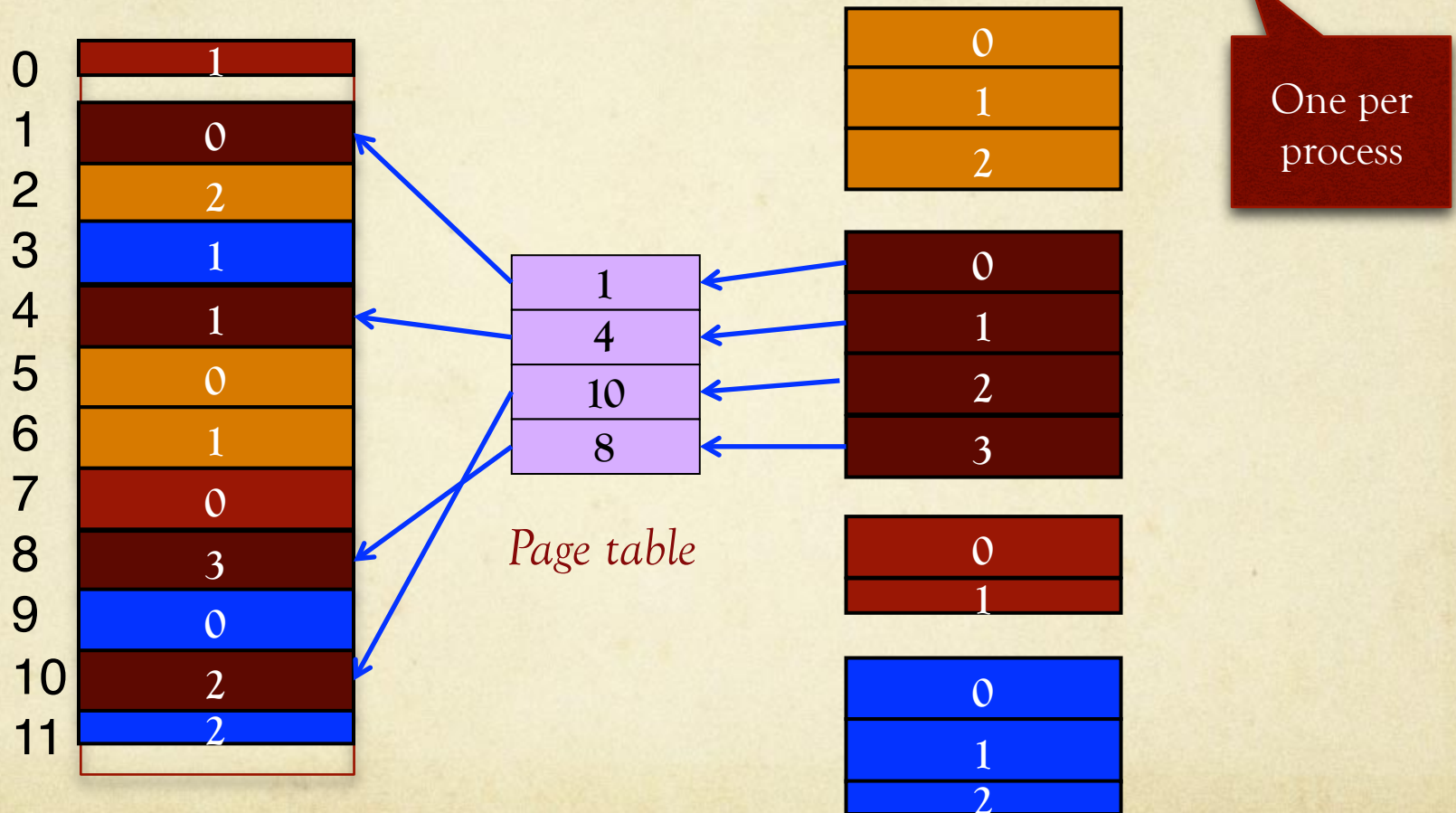
Paging

0	0
1	1
2	2
3	0
4	1
5	2
6	3
7	0
8	1
9	0
10	1
11	2

- ◆ Mapping of process pages could end up being contiguous
- ◆ But it need not be...
- ◆ ...depends on *state* of memory when process is allocated...
- ◆ ...and on *policy* used for allocation

Paging

- OS maintains page to page frame mappings → *page tables*



Paging

- ◆ OS keeps track of *free* page frames → *linked list*
- ◆ To allocate process that needs *n* pages
 - ◆ Find *n* free page frames; *allocate* process pages to frames
- ◆ *Internal* fragmentation possible due to fixed-sized frames
 - ◆ Manageable by carefully choosing *size* of page/page frame

*Part of OS that takes care of mapping,
address translation etc. → memory
management unit*